

What is LLM

An LLM (Large Language Model) is a type of artificial intelligence (AI) designed to understand, process, and generate human-like text. Trained on vast amounts of data—such as books, articles, and websites—these models learn the patterns and relationships between words to perform tasks like answering questions, translating languages, and writing code.

How it Works

- **Transformer Architecture:** LLMs rely on deep learning, utilizing a specialized architecture that processes words in relation to one another to understand context.
- **Pattern Prediction:** Rather than "thinking," they work by predicting what word should come next in a sentence based on the context of the words that came before it.
- **Massive Scale:** The "large" in LLM refers to the billions of data parameters and vast computational power used during training, allowing the AI to grasp complex topics and nuances.

Common Use Cases

- **Content Generation:** Drafting emails, essays, blogs, or creating creative writing.
- **Summarization:** Condensing long documents, articles, or transcripts into quick, easy-to-read points.
- **Coding Assistance:** Writing, debugging, and explaining programming code.
- **Conversational AI:** Powering chatbots and virtual assistants for customer service or general inquiries.

Real-World Examples

- **ChatGPT:** Developed by OpenAI.
- **Gemini:** Developed by Google.
- **Claude:** Developed by Anthropic.

Limitations

While highly capable, LLMs do not possess human reasoning, common sense, or real-world consciousness. Because they predict probabilities, they can sometimes produce incorrect, biased, or completely fabricated information (known as an AI "hallucination").

Major Proprietary Models

- ChatGPT (OpenAI): The most recognizable AI ecosystem, powered by the GPT series. Known for its versatility, vast plugin library, and broad consumer use.
- Claude (Anthropic): A leading conversational and coding assistant known for complex reasoning and its user-friendly interface.
- [Gemini](#) (Google DeepMind): Highly integrated into the Google ecosystem, these models are natively multimodal (capable of processing text, audio, and video). [1,

Popular Open-Weights Models

- Llama (Meta): A foundational family of open-weights models widely used by developers to build custom applications and fine-tune for specific tasks.
- [DeepSeek](#): Chinese open-weight models gaining massive popularity for their highly efficient architecture and advanced mathematical and coding reasoning capabilities.
- Mistral (Mistral AI): A French AI company offering highly capable open-weight models under permissive licenses.

Models

1. Chat / Language Models (LLMs)

These models are designed to understand and generate human language.

♦ What they do:

- Answer questions
- Write code, essays, emails
- Summarize text
- Hold conversations (like ChatGPT)

♦ Examples:

- GPT (OpenAI)
- LLaMA (Meta)
- Claude (Anthropic)

- ◆ **How they work (basic idea):**

They predict the next word in a sentence based on context.

- ◆ **Example:**

User: "What is Python?"

Model: "Python is a high-level programming language used for..."

2. Embedding Models

These models convert text (or other data) into numbers (vectors) that capture meaning.

- ◆ **What they do:**

- Convert text into numerical vectors
- Measure similarity between texts
- Power search engines, recommendation systems, RAG apps

- ◆ **Examples:**

- OpenAI text-embedding models
- Sentence-BERT
- HuggingFace embeddings

- ◆ **Key idea:**

Similar meanings → similar vectors

- ◆ **Example use:**

- "car" and "automobile" will have very close embeddings
-

3. Multimodal Models

These models can understand and process multiple types of data.

- ◆ **What they do:**

- Text + images
- Text + audio
- Text + video

♦ **Examples:**

- GPT-4o (text + image + audio)
- Gemini (Google)
- Claude Vision

♦ **Use cases:**

- Describe images
- Answer questions about charts
- Voice assistants
- Image captioning

♦ **Example:**

You upload an image → model explains what is in it

What is a Prompt?

A **prompt** is the **input or instruction you give to an AI model** to get a response.

Types of prompts

♦ 1. Simple Prompt

A **simple prompt** is a direct, single instruction or question to the AI.

✓ **Example:**

“What is Python?”

or

“Explain REST API.”

📌 **Key idea:**

- No structure
- No role or format
- Just a direct request

Use case:

- Quick answers
 - Basic Q&A
 - Casual usage
-

◆ **2. System + User Prompt**

This is a **two-layer prompt system** used in real AI applications.

✓ **System Prompt (rules/behavior)**

Defines how the AI should behave.

Example:

“You are a helpful coding assistant. Always explain in simple terms.”

✓ **User Prompt (task/input)**

What the user actually asks.

Example:

“Explain Django authentication.”

Combined Flow:

- System sets behavior
- User gives task
- Model responds accordingly

⚡ **Use case:**

- ChatGPT architecture
 - AI apps
 - Chatbots
 - Production systems
-

◆ 3. Prompt Templates

A **prompt template** is a reusable structure with placeholders.

✓ **Example Template:**

“Explain {topic} in simple terms with an example.”

✓ **Filled Example:**

“Explain Docker in simple terms with an example.”

📌 **Key idea:**

- Reusable prompt format
 - Uses variables like {topic}, {language}, {task}
-

⚡ **Use case:**

- RAG systems
 - Automation tools
 - AI pipelines
 - Chatbot frameworks (LangChain, etc.)
-

◆ 4. Structured Prompts

A **structured prompt** organizes input in a clear format (often with sections or JSON-like structure).

✓ Example (section-based):

Task: Summarize text

Tone: Simple and beginner-friendly

Input: AI is transforming the world...

Output format: 3 bullet points

✓ Example (JSON-style):

```
{  
  
  "task": "translate",  
  
  "source_language": "English",  
  
  "target_language": "Nepali",  
  
  "text": "Hello, how are you?"  
}
```

📌 **Key idea:**

- Clearly defined format
 - Reduces confusion
 - Improves accuracy
-

Use case:

- AI APIs
- LLM apps
- RAG systems
- Production-grade AI workflows

What are Chains (in AI / LLMs)?

A **Chain** is a way of **connecting multiple steps or components together** so an AI system can solve a task step-by-step instead of doing everything in one prompt.

In simple words:

👉 A Chain = a workflow of multiple AI operations connected together

What is Memory (in AI / LLM systems)?

Memory in AI means the ability of a model or application to **remember past information** and use it in future responses.

In simple words:

👉 Memory = “AI remembering what happened before in a conversation or system”

◆ Why Memory is needed?

Without memory:

- AI treats every question as new
- No personalization

- No context continuity

With memory:

- AI remembers user preferences
 - Keeps conversation context
 - Gives personalized answers
-

◆ Types of Memory

1. Short-Term Memory (Conversation Memory)

This is memory within a **single chat session**.

✓ Example:

User: "My name is Nishan"

AI: "Nice to meet you Nishan"

User: "What is my name?"

AI: "Your name is Nishan"

📌 Key point:

- Works only in current conversation
 - Lost after session ends (usually)
-

2. Long-Term Memory (Persistent Memory)

This memory is **stored for future sessions**.

✓ Example:

AI remembers:

- Your name
- Your preferences

- Your past projects

 Key point:

- Stored in database or memory store
 - Used across multiple sessions
-

3. Context Window Memory

This is the **temporary memory limit of LLMs**.

✓ **Example:**

- AI can only “see” last 10K–1M tokens (depending on model)
- Older messages are trimmed or summarized

 Key point:

- Built into the model
 - Not permanent storage
-

4. Vector Memory (Embedding-Based Memory)

Used in modern AI apps (RAG systems).

✓ **How it works:**

- Convert text → embeddings
- Store in vector database
- Retrieve similar memory when needed

✓ **Example:**

User asks:

“What did I say about my project?”

System:

- Searches stored embeddings
- Retrieves relevant info
- Sends to LLM

 Used in:

- ChatPDF
 - AI assistants
 - RAG systems
-

5. Working Memory (Active Processing)

This is what AI uses **while thinking step-by-step**.

✓ **Example:**

- Solving math problem step-by-step
- Breaking logic into parts

 Key point:

- Temporary reasoning space
- Not stored long-term

What are Indexes (in AI / Databases / RAG systems)?

An **index** is a structure that helps you **find data faster** instead of scanning everything.

In simple words:

 Index = “shortcut to quickly find information”

◆ Why Indexes are needed?

Without index:

- System searches everything (slow )

With index:

- System jumps directly to relevant data (fast )
-

◆ Real-Life Example

Think of a book:

 **Without index:**

You read every page to find a topic

 **With index:**

You go to the index page → find topic → jump directly

◆ Types of Indexes (Important in AI + DB)

1. Database Index (SQL Index)

Used in relational databases like MySQL, PostgreSQL.

✓ **Example:**

```
CREATE INDEX idx_user_name ON users(name);
```

 What it does:

- Speeds up search queries
 - Avoids full table scan
-

2. Vector Index (AI / Embedding Index)

Used in **RAG systems and embeddings**.

✓ **How it works:**

- Convert text → embeddings (vectors)
- Store in vector DB
- Index helps find similar vectors quickly



Tools:

- FAISS
 - Pinecone
 - Chroma
 - Weaviate
-

✓ **Example:**

User query:

“What is machine learning?”

System:

- Convert to vector
 - Search nearest vectors in index
 - Return most similar documents
-

3. Inverted Index (Search Engines)

Used in Google-like search systems.

✓ **How it works:**

Words → documents mapping

Example:

Word	Documents
------	-----------

AI	Doc1, Doc3
----	------------

Python	Doc2, Doc5
--------	---------------

📌 Used in:

- Elasticsearch
 - Lucene
 - Search engines
-

4. Hash Index

Uses hashing to directly map keys to data.

✓ **Example:**

User ID → Hash → Direct location

📌 Very fast for exact matches

◆ Index in RAG (Very Important 🚀)

In RAG systems:

Flow:

1. Documents → Split into chunks

2. Convert chunks → embeddings
3. Store in vector DB
4. Create **vector index**
5. Query → search index → retrieve relevant chunks



What are Agents (in AI / LLM systems)?

An **AI Agent** is a system that can:

👉 **think + decide + take actions + use tools to complete a task**

In simple words:

🔥 Agent = “LLM + brain + tools + decision making”

◆ Why Agents are needed?

Normal LLM:

- Only responds to prompts
- No real action-taking ability

Agent:

- Can plan steps
 - Can use tools (API, database, web, code)
 - Can make decisions
 - Can complete tasks automatically
-

◆ Simple Example

✗ **Normal LLM:**

“Write Python code for weather API”

It only gives code.

✓ **Agent:**

“Get today’s weather in Kathmandu”

Agent will:

1. Call weather API 🌤️
 2. Fetch data
 3. Process result
 4. Return final answer
-

◆ Core Components of Agents

1. LLM (Brain 🧠)

- Understands task
- Makes decisions

2. Tools 🔧

- APIs
- Database
- Search engine
- Code executor

3. Memory 🧠

- Remembers past interactions

4. Planner 🗺️

- Breaks task into steps

5. Executor ⚙️

- Executes steps using tools

Embedding

Embedding models are machine learning algorithms that translate unstructured data—such as text, images, and audio—into numerical arrays called vectors. By doing so, they map data into a multi-dimensional space where items with similar meanings are positioned close to each other, allowing computers to understand context and semantics.

They are the foundational engine behind semantic search, Retrieval-Augmented Generation (RAG), and recommendation engines.

How They Work

- **Vectorization:** Raw data is encoded into high-dimensional vectors (arrays of numbers).
- **Semantic Capture:** Unlike traditional keyword matching, these numbers capture the underlying meaning, tone, and relationships between concepts.
- **Similarity Scoring:** AI systems compare these vectors using mathematical metrics (like Cosine Similarity or Euclidean Distance) to find the most relevant matches for a query.